

Dockerized Sports Rust Service

Manufacturing Operational Insights Report

Course Name: DevOps Foundation

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	APARNA SHARMA	EN24CA5030023
2.	SHUBHANJALI ROY	EN24CA5030161
3.	SANDHYA PATIDAR	EN24CA5030150
4.	MIMANSA KASHYAP	EN22IT301054
5.	HARSH GUPTA	EN24CA5030057
6.	ADITYA BAIRAGI	EN24CA5030009

Group Name: Group 12D12

Project Number: DO-50

Industry Mentor Name: Vaibhav

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features (*Just mention key features here no need to go into details*)
2. Overall Architecture / Workflow
3. X`x`)

Results & Output

Add the below details here:

1. Screenshots / outputs
2. Reports / dashboards / models
3. Key outcomes

Conclusion

Mention a brief conclusion about your project summing up everything you have worked on and the key learning.

Future Scope & Enhancemen

Table of Contents

Page No.
1. Introduction
2. Problem Statement & Objectives
2.1 Problem Statement
2.2 Project Objectives
2.3 Scope of the Project
3. Proposed Solution
3.1 Key features
3.2 Overall Architecture / Workflow
3.3 Tools & Technologies Used
4. Results & Output
4.1 Screenshots / outputs
4.2 Reports / dashboards / models
4.3 Key outcomes
5. Conclusion
6. Future Scope & Enhancements

1.Introduction

Modern software development increasingly relies on containerization to ensure consistency, scalability, and portability across different environments. Docker has become a standard tool for packaging applications and their dependencies into lightweight, reproducible containers, reducing environment-related issues and improving deployment reliability.

This project focuses on containerizing a **Sports Service built with Rust**, applying industry best practices to create an efficient, secure, and production-ready setup. A multi-stage Dockerfile is used to minimize image size, improve build performance, and reduce the attack surface by separating the build and runtime environments. In addition, Docker Compose is employed to orchestrate the Rust application alongside its required database services, enabling seamless local development and simplified service management.

By leveraging Docker and Docker Compose, this solution provides a consistent development and deployment workflow for the entire team, ensuring that the application behaves identically across development, testing, and production environments. The approach enhances maintainability, improves collaboration, and lays a strong foundation for future scalability and deployment automation.

2. Problem Statement & Objectives

2.1 Problem Statement

The Sports Service application, developed using the Rust programming language, is designed to deliver high performance, reliability, and safety. However, deploying and maintaining such an application across multiple environments presents several challenges. Differences in operating systems, library versions, system configurations, and runtime dependencies often lead to inconsistencies between development, testing, and production environments. These inconsistencies can cause unexpected application behavior, deployment failures, and increased debugging time, ultimately slowing down the development process.

Furthermore, deploying a Rust-based service without an optimized containerization strategy may result in large container images, inefficient resource utilization, and longer build and deployment times. From a security perspective,

improperly configured

containers can expose unnecessary system components, increasing the application's attack

surface. As the application also depends on external services such as a database, managing these dependencies manually or through ad-hoc scripts becomes complex and error-prone, especially when multiple developers or environments are involved.

Another significant challenge is ensuring seamless collaboration within a development team. Without a standardized environment, each team member may require extensive setup and configuration, leading to onboarding difficulties and inconsistencies in application behavior. Scaling the application or preparing it for production deployment further amplifies these issues when the deployment process is not automated or standardized.

The core problem addressed in this project is the need for a robust, secure, and efficient containerization approach for the

Sports Service. This involves designing multi-stage Dockerfiles that separate the build and runtime environments to optimize image size, improve performance, and enhance security. Additionally, Docker Compose is used to orchestrate the Rust application alongside its database dependencies, enabling consistent service management and environment replication. The objective is to establish a portable, reproducible, and production-ready infrastructure that simplifies development, improves reliability, and ensures consistent behavior across all stages of the software lifecycle.

2.2 Project Objectives

The primary objectives of this project are as follows:

I. To Containerize the Sports Rust Service

The main objective is to containerize the Sports Service developed in Rust using Docker. This ensures that the application and its dependencies are packaged together, allowing it to run consistently across different environments without requiring manual configuration.

II. To Implement Multi-Stage Docker Builds

This project aims to design and implement multi-stage Dockerfiles that separate the build environment from the runtime environment. This approach helps reduce the final image size, improve build efficiency, and minimize security risks by excluding unnecessary build tools from the production image.

III. To Optimize Image Size and Performance

Another key objective is to optimize the Docker image for performance and resource usage. By using lightweight

base images and efficient build strategies, the project seeks to achieve faster container startup times and reduced storage and network overhead.

IV. To Enhance Application Security

The project focuses on following Docker security best practices, such as minimizing the attack surface, running the application with limited privileges, and avoiding unnecessary dependencies in the final image. This helps ensure a more secure and production-ready deployment.

V. To Orchestrate Services Using Docker Compose

The project aims to configure Docker Compose to manage the Sports Service along with its database dependencies. This allows multiple services to be started, stopped, and managed together, simplifying local development and testing.

VI. To Ensure Environment Consistency Across the Team

A critical objective is to provide a standardized

development and deployment environment for all team members. Docker and Docker Compose help eliminate environment-specific issues, improving collaboration and reducing onboarding time for new developers.

VII. To Simplify Development, Testing, and Deployment Workflows

By using containerization and orchestration tools, the project seeks to streamline the overall software lifecycle. Developers can easily replicate production-like environments locally, leading to more reliable testing and smoother deployments.

VIII. To Build a Scalable and Maintainable Deployment Setup

The final objective is to establish a foundation that supports future scalability and maintenance. The Docker-based setup can be easily extended to include additional

services, monitoring tools, or CI/CD pipelines as the project evolves.

2.3 Scope of the Project

The scope of this project is focused on the containerization and orchestration of a Sports Service application developed using the Rust programming language. The project primarily covers the design, implementation, and configuration of Docker-based solutions to ensure a consistent, secure, and portable runtime environment for the application.

This project includes the creation of a multi-stage Dockerfile to build and package the Rust-based Sports Service efficiently. The build stage is responsible for compiling the application and managing dependencies, while the runtime stage contains only the essential components required to execute the service. This approach helps reduce image size, improve performance, and enhance security by limiting the attack surface.

The scope also includes the use of Docker Compose to orchestrate the Sports Service along with its database dependencies. Docker Compose is used to define and

manage multiple services,

enabling seamless startup, shutdown, and communication between the application and the database. This allows developers to replicate production-like environments locally with minimal effort.

Additionally, the project focuses on adopting industry best practices related to Docker image optimization, security, and maintainability. This includes using lightweight base images, following proper layering strategies, and ensuring environment consistency across development and deployment stages. The setup is designed to support team collaboration by providing a standardized environment that can be easily shared and reproduced.

However, this project does not cover advanced deployment scenarios such as container orchestration using Kubernetes, cloud-specific deployment configurations, or CI/CD pipeline automation. Performance benchmarking, load testing, and production monitoring are also considered outside the scope of this project. The primary emphasis remains on establishing

a reliable and production-ready containerization framework for the Sports Rust Service.

3. Proposed Solution

3.1 Key features

The key features of the Dockerized Sports Rust Service project are outlined below:

I. **Rust-Based High-Performance Service**

The Sports Service is developed using the Rust programming language, which is known for its high performance, memory safety, and reliability. This ensures efficient handling of application logic while minimizing runtime errors and security vulnerabilities.

II. **Multi-Stage Dockerfile Implementation**

The project utilizes multi-stage Docker builds to separate the build environment from the runtime environment.

This significantly reduces the final container image size and ensures that only the necessary binaries and dependencies are included in the production image.

III. Optimized and

Lightweight Container Images

Lightweight base images and efficient build strategies are used to create optimized Docker images. This leads to faster build times, reduced storage usage, and quicker container startup, making the application more efficient and scalable.

IV. Secure Container Configuration

The container setup follows Docker security best practices by minimizing the attack surface, avoiding unnecessary tools in the runtime image, and ensuring a clean and secure execution environment suitable for production use.

V. Service Orchestration with Docker Compose

Docker Compose is used to orchestrate the Sports Service and its database dependencies. This enables multiple services to be managed together through a single configuration file, simplifying development, testing, and deployment workflows.

VI. Environment Consistency Across Development and Deployment

The Docker-based setup ensures that the application runs consistently across all environments, including development, testing, and production. This eliminates environment-specific issues and improves collaboration among team members.

VII. Simplified Setup and Onboarding

New developers can quickly set up the project using Docker and Docker Compose without manually installing dependencies. This reduces onboarding time and ensures a smooth development experience.

VIII. Production-Ready and Maintainable Design

The overall design emphasizes maintainability and readiness for production. The containerization approach allows easy updates, scalability, and future

enhancements without major
changes to the existing setup.

3.2 Overall Architecture / Workflow

The overall architecture of the project is based on a containerized, service-oriented design that ensures modularity, portability, and consistency across environments. The Sports Service application, developed using Rust, is packaged and deployed using Docker, while Docker Compose is used to manage the interaction between the application and its database dependencies.

Architecture Overview

The architecture consists of the following main components:

Sports Rust Service

The core application is a Rust-based service that contains the business logic related to sports data processing and management. This service is containerized using a multi-stage Dockerfile to ensure an optimized and secure runtime environment.

Database Service

A database service is used to store and manage persistent application data. This service runs in a separate container and communicates with the Sports Service over a Docker-managed network. The separation of concerns improves scalability and maintainability.

Docker Engine

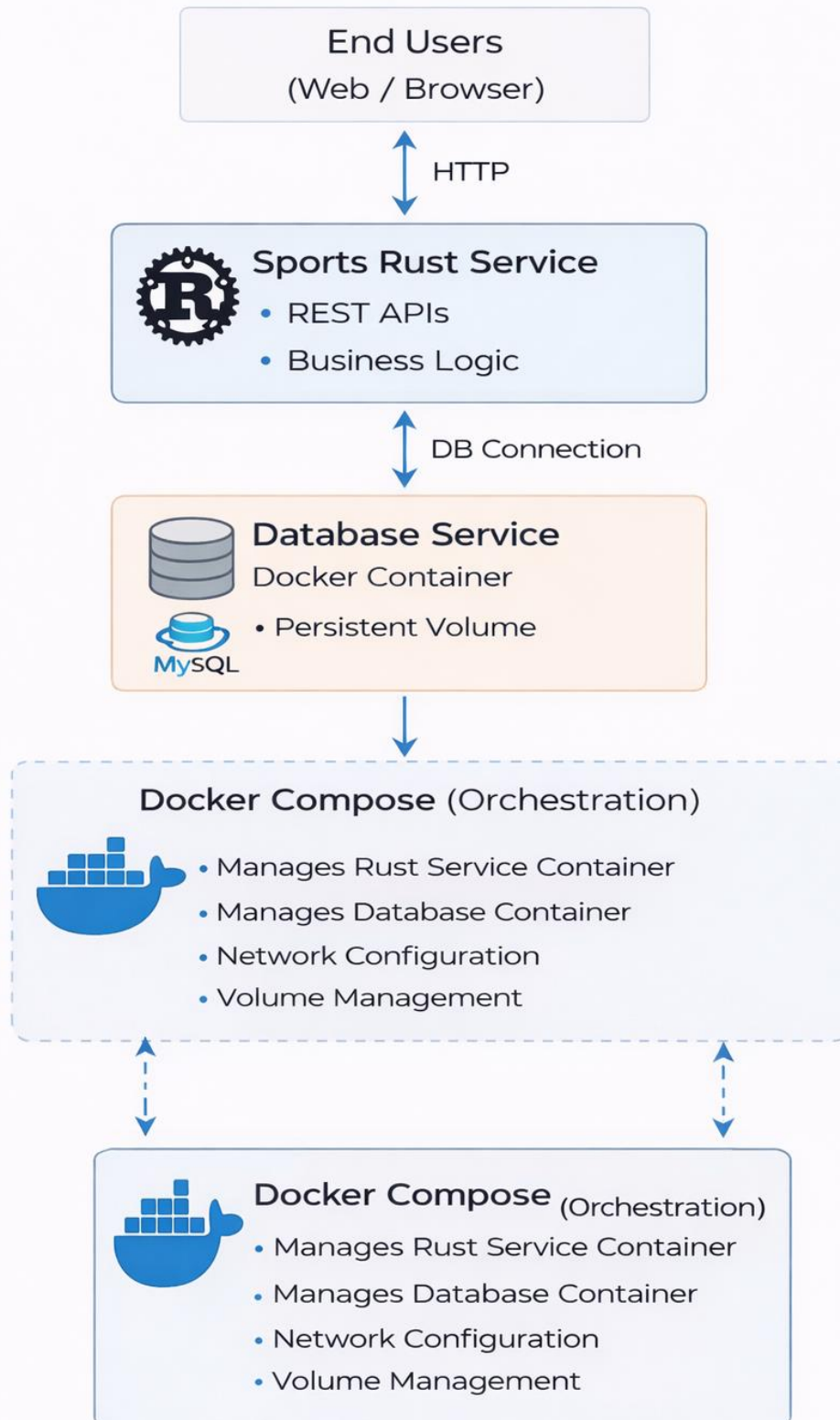
Docker is responsible for building container images and running containers in isolated environments. It ensures that the application behaves consistently regardless of the underlying operating system.

Docker Compose

Docker Compose acts as the orchestration layer, defining all services, networks, and volumes in a single configuration file. It enables easy startup, shutdown, and

management of the entire
application stack using simple commands.

Overall System Architecture



Workflow

Application Development

Developers write and update the Sports Service code in Rust on their local machines. Changes are reflected in the containerized environment without requiring manual dependency installation.

Docker Image Build Process

The multi-stage Dockerfile first compiles the Rust application in a build stage. Once the binary is successfully built, only the compiled executable and essential runtime files are copied into the final runtime image.

Service Orchestration

Docker Compose is used to start the Sports Service and the database service together. It ensures proper service dependency management, network configuration, and environment variable handling.

Inter-Service Communication

The Sports Service communicates with the database

through an internal Docker network. This setup isolates internal communication and enhances security.

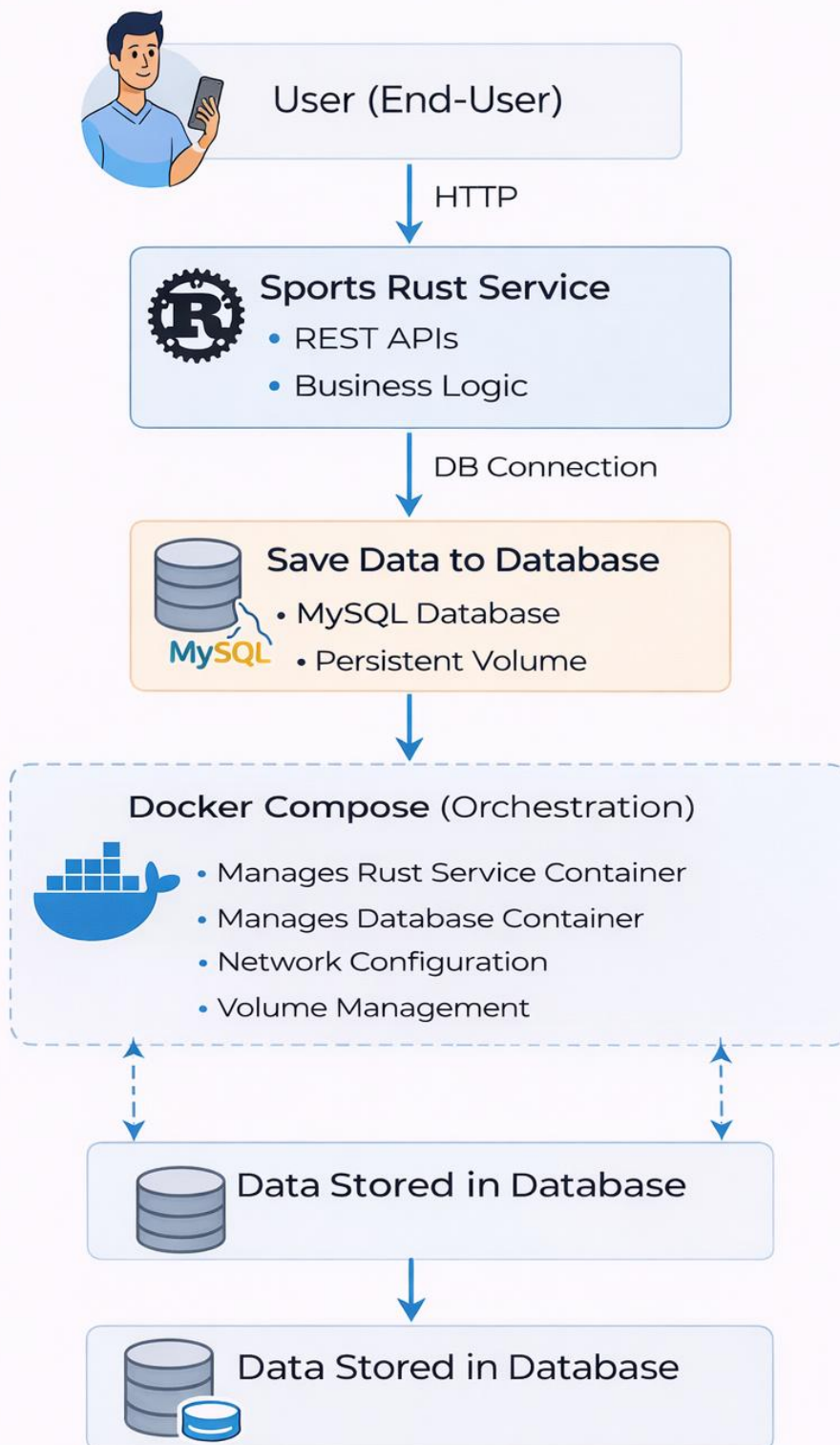
Application Execution

Once all services are running, the Sports Service becomes accessible through the exposed ports defined in the Docker Compose configuration.

Consistency Across Environments

The same Docker and Docker Compose configuration is used across development, testing, and production environments, ensuring predictable behavior and reducing deployment-related issues.

Sports Rust Service Workflow



3.3 Tools & Technologies Used

Tools

Git

Git is used as a version control system to manage the source code of the Sports Rust Service. It helps track changes, maintain code history, and enables smooth collaboration among team members by supporting branching and merging workflows.

Docker

Docker is used to containerize the Sports Rust Service and its database dependencies. It ensures consistent environments across development, testing, and production by packaging the application along with all required dependencies.

Jenkins / GitHub Actions

Jenkins or GitHub Actions are used for continuous integration and continuous deployment (CI/CD). These tools automate the process of building Docker images, running tests, and

deploying the application, thereby reducing manual effort and improving reliability.

Kubernetes (K8s)

Kubernetes is used for container orchestration in advanced deployment scenarios. It helps manage container scaling, load balancing, service discovery, and high availability for the Sports Service in production environments.

Technologies

HTML

HTML is used to create the basic structure of the web interface that allows users to interact with the Sports Service through a browser.

CSS

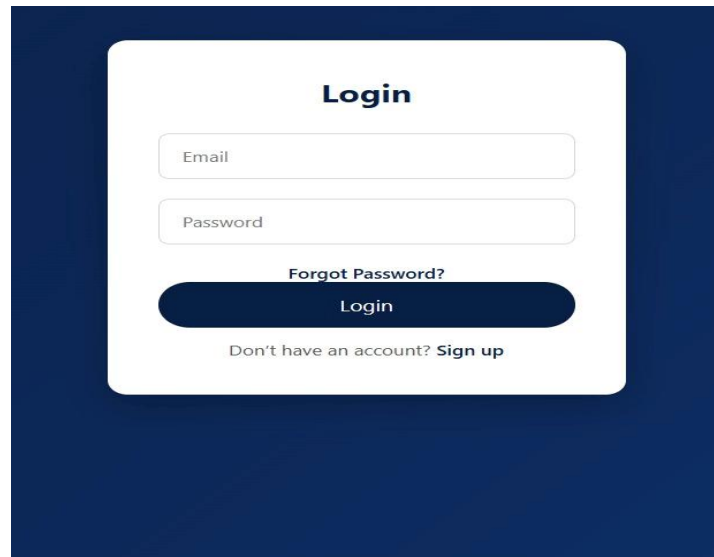
CSS is used for styling and designing the user interface, improving the visual appearance and user experience of the web application.

React

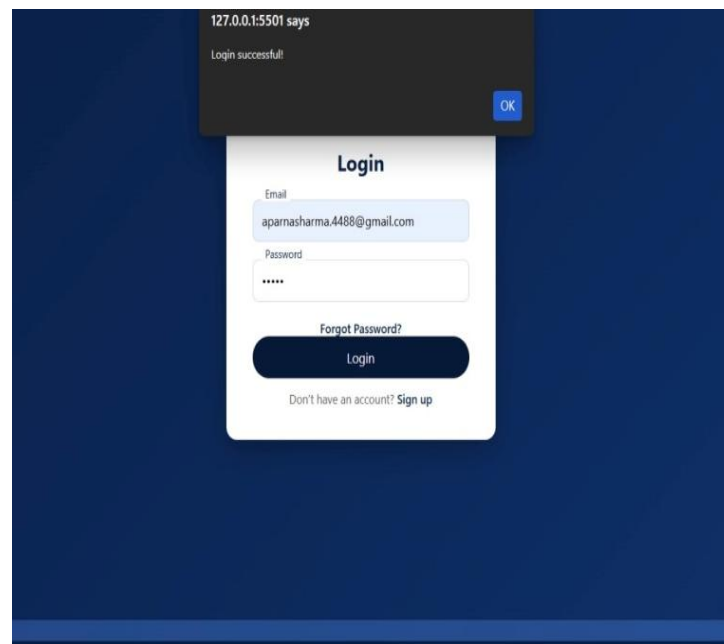
React is used to build a dynamic and interactive front-end interface. It enables component-based development, efficient UI updates, and seamless integration with the backend Sports Rust Service through REST APIs.

4. Results & Output

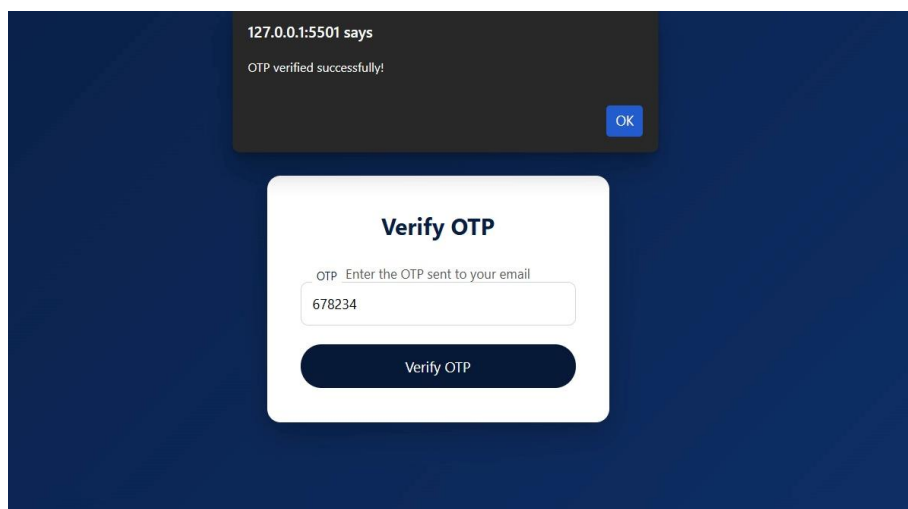
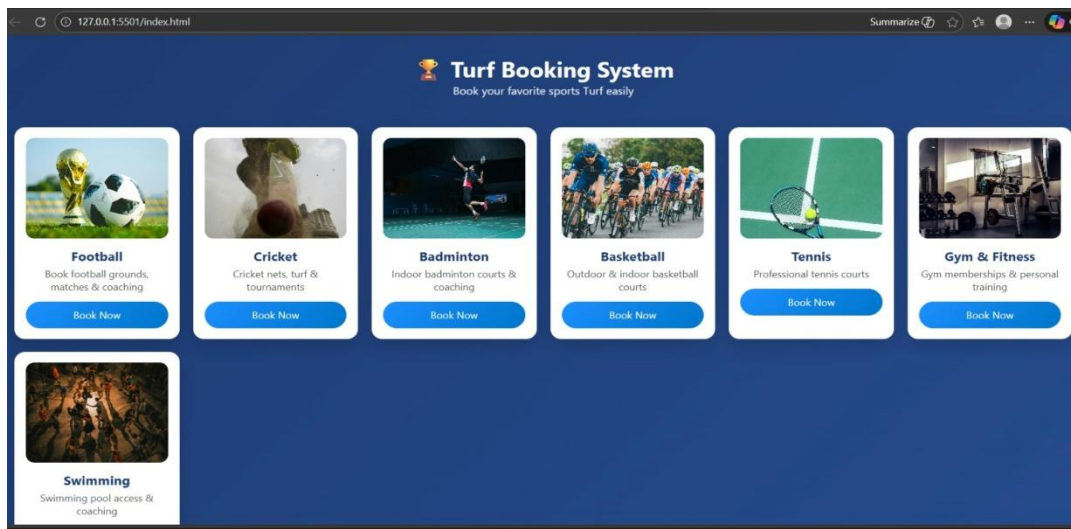
4.1 Screenshots / outputs

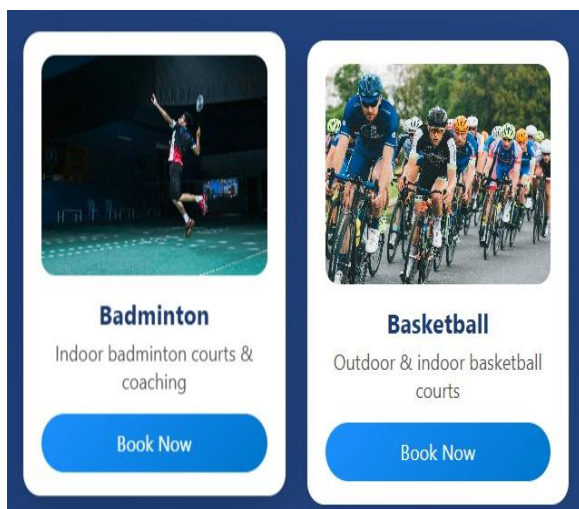


A screenshot of a login form on a dark blue background. The form is white and contains the following elements: a title "Login", an "Email" input field, a "Password" input field, a "Forgot Password?" link, a dark blue "Login" button, and a "Don't have an account? Sign up" link.



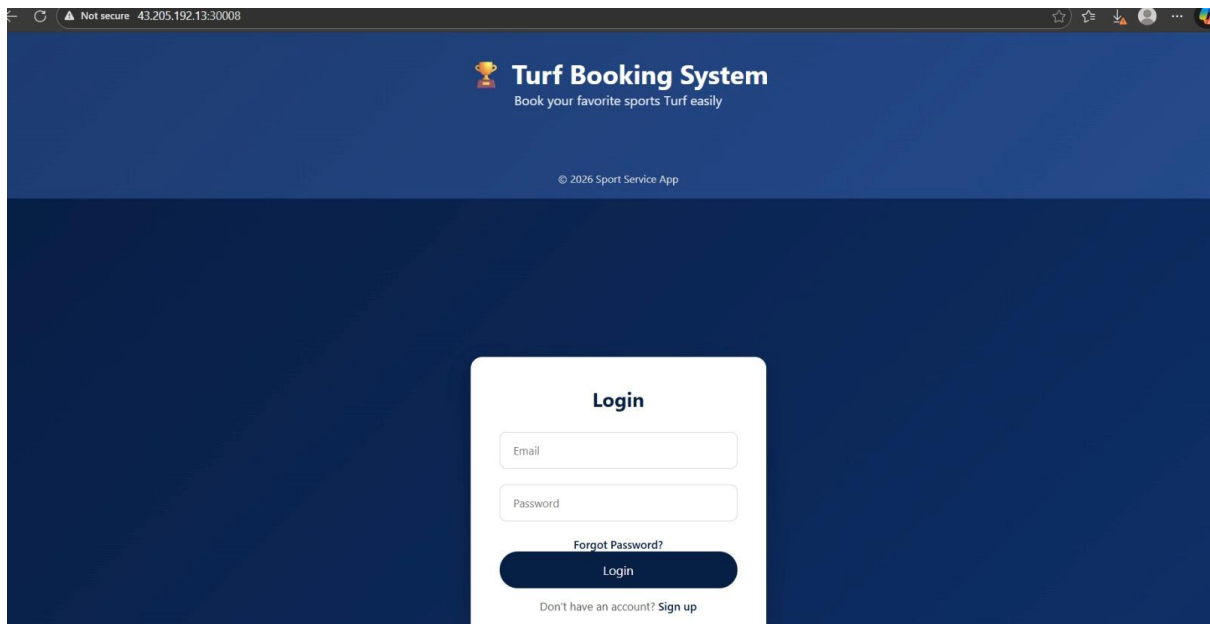
A screenshot of the same login form, but with a success message overlay. The message is "127.0.0.1:5501 says Login successful!" and has an "OK" button. The form fields are now populated: the "Email" field contains "aparnasharma.4488@gmail.com" and the "Password" field contains "*****". The "Login" button is still visible.





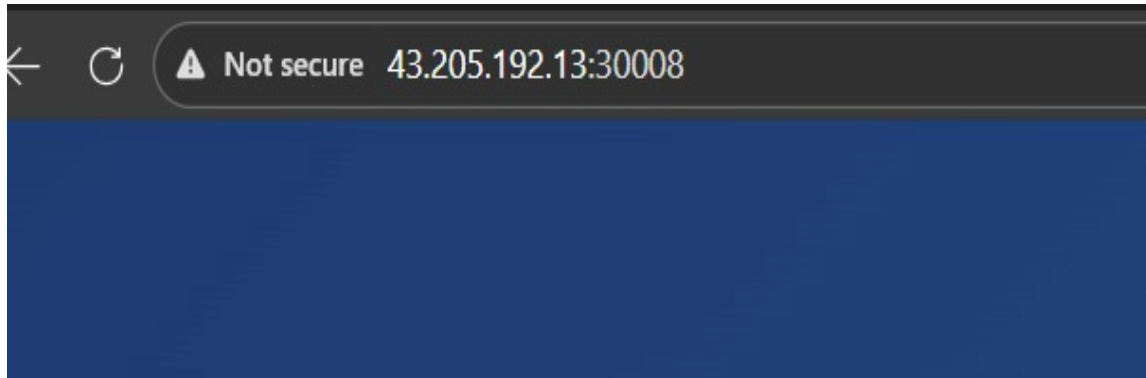
4.2 Reports / dashboards / models

```
PodScheduled: True
Volumes:
  kube-api-access-zbgcq:
    Type: Projected (a volume that contains injected data from mul
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional: false
    DownwardAPI: true
QoS Class: BestEffort
Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300
              node.kubernetes.io/unreachable:NoExecute op=Exists for 300
Events:
  Type     Reason      Age   From          Message
  ----     ------      -
  Normal   Scheduled   31s   default-scheduler   Successfully assigned default/sports-de
  Normal   Pulling     31s   kubelet         Pulling image "jezash/sports-service:v2
  Normal   Created     30s   kubelet         Created container: sports-container
  Normal   Started     30s   kubelet         Started container sports-container
ubuntu@ip-172-31-47-145:~/Sports-service$ sudo kubectl apply -f k8s/deployment.yaml
deployment.apps/sports-deployment configured
ubuntu@ip-172-31-47-145:~/Sports-service$ sudo kubectl rollout restart deployment sp
deployment.apps/sports-deployment restarted
ubuntu@ip-172-31-47-145:~/Sports-service$ sudo kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
sports-deployment-7cb7ffdc5-xk6z8   1/1     Running   0           15s
```



i-03bec3be48f19d699 (Sports-Serv

PublicIPs: 43.205.192.13 PrivateIPs: 172.31.4



4.3 Key outcomes

The successful implementation of the Dockerized Sports Rust Service resulted in several important outcomes that demonstrate the effectiveness of the proposed solution.

Consistent and Portable Deployment Environment

The project achieved a fully containerized setup that ensures the Sports Service runs consistently across development, testing, and production environments. This eliminated environment-specific issues and improved deployment reliability.

Optimized and Lightweight Docker Images

By using multi-stage Docker builds, the final container images were significantly reduced in size. This led to faster build times, improved performance, and lower resource consumption.

Improved Security and

Isolation

The containerized architecture enhanced application security by isolating the Sports Service and database into separate containers and minimizing unnecessary dependencies in the runtime environment.

Efficient Service Orchestration

Docker Compose enabled smooth orchestration of the Sports Rust Service and its database dependencies. It simplified service management, networking, and volume handling through a single configuration file.

Simplified Development and Team Collaboration

The standardized Docker-based setup reduced onboarding time for new developers and ensured all team members worked in identical environments, improving collaboration and productivity.

Scalable and Maintainable Architecture

The project established a strong foundation for scalability

and future enhancements. The containerized design allows easy extension to CI/CD pipelines and orchestration platforms like Kubernetes.

Production-Ready Infrastructure

The overall solution is production-ready, following industry best practices for containerization, security, and maintainability, making it suitable for real-world deployment scenarios.

5. Conclusion

This project successfully demonstrates the containerization of a Sports Service developed using the Rust programming language by applying industry best practices with Docker and Docker Compose. The implementation of multi-stage Dockerfiles resulted in optimized, lightweight, and secure container images, improving performance and reducing resource consumption. By orchestrating the application and its database dependencies using Docker Compose, the project ensured consistent service management and seamless inter-container communication.

The containerized architecture eliminated environment-related inconsistencies and provided a standardized development and deployment workflow for the entire team. This not only simplified application setup and onboarding but also improved collaboration and reliability across different stages of the software lifecycle. Overall, the project delivered a portable, scalable, and production-ready solution that aligns with

modern DevOps practices and lays a strong foundation for future deployment and automation strategies.

6.Future Scope & Enhancements

Although the project meets its primary objectives, there are several areas where further enhancements can be made to extend its functionality and scalability:

Kubernetes-Based Deployment

The Dockerized setup can be extended to Kubernetes for advanced container orchestration, enabling features such as auto-scaling, load balancing, self-healing, and high availability.

CI/CD Pipeline Automation

Integration with CI/CD tools like Jenkins or GitHub Actions can be enhanced to automate testing, image building, security scanning, and deployment processes, further improving development efficiency.

Cloud Deployment

The application can be deployed on cloud platforms such

as AWS, Azure, or Google Cloud to leverage managed services, scalability, and global availability.

Monitoring and Logging

Implementing monitoring and logging tools like Prometheus, Grafana, or the ELK stack would improve system observability and help in performance analysis and troubleshooting.

Security Enhancements

Additional security measures such as secrets management, vulnerability scanning, and role-based access control (RBAC) can be introduced to strengthen the application's security posture.

Performance Optimization and Load Testing

Future enhancements may include performance benchmarking and load testing to evaluate system behavior under high traffic and optimize resource usage accordingly.

Feature Expansion

New features related to sports analytics, user management, or real-time data processing can be added without major architectural changes due to the modular container-based design.